Informe de pruebas

Sistema inteligente basado en reglas lógicas y búsqueda heurística para encontrar la mejor ruta en el MIO (Cali)

Curso	Inteligencia Artificial
Actividad	Actividad 2 - Búsqueda y sistemas basados en reglas
Tutor	Sandra Bautista
Institución	Corporación Universitaria Iberoamericana
Integrantes	José Arias Juan Sebastián Gómez Torres
Fecha	27/09/2026

1. Objetivo y alcance

Verificar que el sistema (a) represente correctamente el conocimiento del sistema de transporte masivo mediante hechos y reglas lógicas, (b) derive por inferencia el conocimiento necesario (conexiones bidireccionales, transbordos, costos, disponibilidad) y (c) encuentre la mejor ruta entre dos estaciones usando búsqueda heurística A*, comparándola con otras estrategias.

Datos: la red se construyó con datos reales del MIO descargados del Buscarutas del SIUR (servicios.siur.com.co/buscarutas): 81 estaciones y 9 líneas (T31, T40, T47, T50, T57, T62, T12, E21, E37) con sus paradas en el orden real del recorrido. Los tiempos se estiman con la distancia sobre el recorrido y una velocidad comercial supuesta (20 km/h troncal, 24 km/h expreso).

2. Entorno de ejecución

Python 3.13.14 | Sistema operativo: win32 | Sin dependencias externas para el sistema (solo biblioteca estándar). Comando de pruebas: `python -m unittest discover -s tests -v`

3. Pruebas automáticas (unittest)

Resultado global: **27 de 27 pruebas aprobadas**.

#	Módulo	Prueba / qué verifica	Resultado
1	busqueda	**test_astar_expande_menos_nodos** En total, A* expande al menos 25 % menos nodos que costo uniforme.	**APROBADA**
2	busqueda	**test_astar_optimo_en_todos_los_pares** A* obtiene el mismo tiempo óptimo que costo uniforme en los 6480 pares.	**APROBADA**
3	busqueda	**test_criterio_menos_transbordos** Aguablanca -> Univalle: por tiempo 57.6 min con 2 transbordos; por transbordos 63.1 min con 1.	**APROBADA**
4	busqueda	**test_estacion_desconocida** Una estación inexistente produce un error claro.	**APROBADA**
5	busqueda	**test_heuristica_admisible** h(n) nunca sobreestima el costo real (comparado con el óptimo de costo uniforme).	**APROBADA**
6	busqueda	**test_misma_estacion** Si origen y destino coinciden, la ruta tiene 0 pasos y 0 minutos.	**APROBADA**
7	busqueda	**test_red_conectada** Existe ruta entre todos los pares de las 81 estaciones (6480 pares).	**APROBADA**
8	busqueda	**test_ruta_andres_sanin_universidades** A* encuentra T40 hasta San Pascual + transbordo al expreso E21 (58.3 min).	**APROBADA**
9	busqueda	**test_voraz_y_bfs_no_garantizan_optimo** Cañaverales -> Popular: voraz (88.4) y BFS (82.1) son mucho más lentas que A* (51.3).	**APROBADA**
10	busqueda	**test_cli_comparar** El comando 'comparar' muestra los cuatro algoritmos.	**APROBADA**
11	busqueda	**test_cli_consultar** El comando 'consultar' responde consultas lógicas.	**APROBADA**
12	busqueda	**test_cli_ruta** El comando 'ruta' imprime el itinerario y el tiempo total.	**APROBADA**
13	busqueda	**test_estacion_cerrada_cambia_ruta** Con San Pascual cerrada, el sistema cambia el transbordo a San Bosco (T47 + E21).	**APROBADA**
14	busqueda	**test_ruta_suspendida** Si se suspenden T40 y T47, Andrés Sanín queda sin servicio y no hay ruta.	**APROBADA**

#	Módulo	Prueba / qué verifica	Resultado
15	logica	**test_alcanzabilidad_recursiva** Reglas R12-R13: toda estación de la red es alcanzable desde Menga.	**APROBADA**
16	logica	**test_costo_transbordo_terminal** Reglas R8-R9: transbordar en una terminal cuesta menos que en una estación.	**APROBADA**
17	logica	**test_datos_reales** La base generada desde el SIUR contiene 81 estaciones y 9 rutas reales del MIO.	**APROBADA**
18	logica	**test_espera** Regla R10: la espera promedio es la mitad de la frecuencia.	**APROBADA**
19	logica	**test_estacion_cerrada** Reglas R3-R4: con Centro cerrada el T40 no se detiene allí, pero pasa de largo (San Pascual -> La Ermita).	**APROBADA**
20	logica	**test_explicacion** El sistema muestra el árbol de prueba de un hecho derivado.	**APROBADA**
21	logica	**test_tramos_dirigidos** Regla R1: los tramos son dirigidos, en el sentido real del recorrido (T40 va de Centro a La Ermita, no al revés).	**APROBADA**
22	logica	**test_transbordos_en_centro** Regla R6: en Centro se puede transbordar entre las 6 rutas que lo sirven.	**APROBADA**
23	logica	**test_encadenamiento_hacia_adelante** Se derivan hechos nuevos aplicando reglas encadenadas y recursivas.	**APROBADA**
24	logica	**test_incorporados** Los predicados incorporados distinto/2 y mitad/2 funcionan dentro de reglas.	**APROBADA**
25	logica	**test_negacion_por_fallo** La negación (\+) se evalúa después de completar el estrato del que depende.	**APROBADA**
26	logica	**test_error_sintaxis** Una cláusula mal escrita produce ErrorSintaxis.	**APROBADA**
27	logica	**test_hechos_y_reglas** El analizador distingue hechos de reglas y reconoce números, cadenas y comentarios.	**APROBADA**

4. Escenarios de uso ejecutados

Cada escenario muestra el comando ejecutado y la salida real obtenida.

E1. Ruta óptima con A* (caso base)

Desde la Terminal Andrés Sanín hasta la Terminal Universidades. El nombre se escribe sin tildes para verificar la resolución de nombres.

```
$ python main.py ruta "Andres Sanin" Universidades --detalle

Mejor ruta de Andrés Sanín a Universidades
Algoritmo: A* | Criterio: tiempo
----------------------------------------------------------------------
1. En Andrés Sanín: Aborde la ruta T40 (espera/transbordo 4 min). Viaje 8 parada(s) hasta San Pascual (17.6
    min).
     Paradas: 7 de Agosto -> El Trébol -> Villacolombia -> Chapinero -> Atanasio Girardot -> Floresta ->
         Belalcázar -> San Pascual
2. En San Pascual: Haga transbordo a la ruta E21 (espera/transbordo 10 min). Viaje 11 parada(s) hasta
    Universidades (26.7 min).
     Paradas: San Bosco -> Santa Librada -> Estadio -> Tequendama -> Unidad Deportiva -> Pampalinda -> Caldas ->
          Capri -> Meléndez -> Buitrera -> Universidades
Tiempo total estimado: 58.3 min | Transbordos: 1 | Paradas recorridas: 19
Nodos expandidos: 106 | generados: 154 | 0.73 ms

Secuencia de estados (estación, ruta):
  abordar     ('andres_sanin', None) -> ('andres_sanin', 't40')  (+4 min)
  viajar      ('andres_sanin', 't40') -> ('e_7_de_agosto', 't40')  (+3.9 min)
  viajar      ('e_7_de_agosto', 't40') -> ('el_trebol', 't40')  (+3.1 min)
  viajar      ('el_trebol', 't40') -> ('villacolombia', 't40')  (+1 min)
  viajar      ('villacolombia', 't40') -> ('chapinero', 't40')  (+1.1 min)
  viajar      ('chapinero', 't40') -> ('atanasio_girardot', 't40')  (+2 min)
  viajar      ('atanasio_girardot', 't40') -> ('floresta', 't40')  (+2.1 min)
  viajar      ('floresta', 't40') -> ('belalcazar', 't40')  (+2.1 min)
  viajar      ('belalcazar', 't40') -> ('san_pascual', 't40')  (+2.3 min)
  transbordo  ('san_pascual', 't40') -> ('san_pascual', 'e21')  (+10 min)
  viajar      ('san_pascual', 'e21') -> ('san_bosco', 'e21')  (+1.6 min)
  viajar      ('san_bosco', 'e21') -> ('santa_librada', 'e21')  (+1.5 min)
  viajar      ('santa_librada', 'e21') -> ('estadio', 'e21')  (+2.8 min)
  viajar      ('estadio', 'e21') -> ('tequendama', 'e21')  (+2.4 min)
  viajar      ('tequendama', 'e21') -> ('unidad_deportiva', 'e21')  (+2.5 min)
  viajar      ('unidad_deportiva', 'e21') -> ('pampalinda', 'e21')  (+3.1 min)
  viajar      ('pampalinda', 'e21') -> ('caldas', 'e21')  (+2.8 min)
  viajar      ('caldas', 'e21') -> ('capri', 'e21')  (+1.7 min)
  viajar      ('capri', 'e21') -> ('melendez', 'e21')  (+3 min)
  viajar      ('melendez', 'e21') -> ('buitrera', 'e21')  (+1.5 min)
  viajar      ('buitrera', 'e21') -> ('universidades', 'e21')  (+3.8 min)
```

E2. Comparación de estrategias de búsqueda

Cañaverales -> Popular. Se comparan A*, costo uniforme, voraz y amplitud.

```
$ python main.py comparar canaverales popular

Comparación de estrategias: Cañaverales -> Popular

Algoritmo         Minutos  Transb.  Paradas  Expand.  Gener.      ms
----------------------------------------------------------------------
A*                  51.3        1       20      123     170    0.79 *
Costo uniforme      51.3        1       20      160     214    1.00 *
Voraz (greedy)      88.4        3       32       38      57    0.22
Amplitud (BFS)      82.1        3       17      160     168    0.45

* = menor tiempo encontrado. A* y costo uniforme siempre son óptimos;
  A* expande menos nodos gracias a la heurística admisible.
```

E3. Estación cerrada (hecho agregado en tiempo de ejecución)

Mismo viaje del E1 con la estación San Pascual cerrada: por la regla R4 los buses pasan sin detenerse y el sistema traslada el transbordo a San Bosco.

```
$ python main.py ruta andres_sanin universidades --cerrar "San Pascual"

Mejor ruta de Andrés Sanín a Universidades
Algoritmo: A* | Criterio: tiempo
Estaciones cerradas: San Pascual
----------------------------------------------------------------------
1. En Andrés Sanín: Aborde la ruta T47 (espera/transbordo 4 min). Viaje 8 parada(s) hasta San Bosco (19.5 min).
     Paradas: 7 de Agosto -> El Trébol -> Villacolombia -> Chapinero -> Atanasio Girardot -> Floresta ->
         Belalcázar -> San Bosco
2. En San Bosco: Haga transbordo a la ruta E21 (espera/transbordo 10 min). Viaje 10 parada(s) hasta
    Universidades (25.1 min).
     Paradas: Santa Librada -> Estadio -> Tequendama -> Unidad Deportiva -> Pampalinda -> Caldas -> Capri ->
         Meléndez -> Buitrera -> Universidades
Tiempo total estimado: 58.6 min | Transbordos: 1 | Paradas recorridas: 18
Nodos expandidos: 98 | generados: 144 | 0.84 ms
```

E4a. Criterio: menor tiempo

Aguablanca -> Univalle minimizando el tiempo total.

```
$ python main.py ruta aguablanca univalle

Mejor ruta de Aguablanca a Univalle
Algoritmo: A* | Criterio: tiempo
----------------------------------------------------------------------
1. En Aguablanca: Aborde la ruta T50 (espera/transbordo 4 min). Viaje 4 parada(s) hasta Calipso (9.5 min).
     Paradas: Nuevo Latir -> Amanecer -> Troncal Unida -> Calipso
2. En Calipso: Haga transbordo a la ruta T12 (espera/transbordo 9 min). Viaje 9 parada(s) hasta Universidades
    (25.3 min).
     Paradas: El Diamante -> Antonio Nariño -> Ciudad Modelo -> República de Israel -> Villa del Sur -> Mariano
         Ramos -> Cañaverales -> Simón Bolívar -> Universidades
3. En Universidades: Haga transbordo a la ruta T31 (espera/transbordo 7 min). Viaje 1 parada(s) hasta Univalle
    (2.8 min).
Tiempo total estimado: 57.6 min | Transbordos: 2 | Paradas recorridas: 14
Nodos expandidos: 118 | generados: 163 | 0.88 ms
```

E4b. Criterio: menos transbordos

El mismo viaje minimizando primero los transbordos: se aceptan 5.5 minutos más a cambio de un cambio de bus menos.

```
$ python main.py ruta aguablanca univalle --criterio transbordos

Mejor ruta de Aguablanca a Univalle
Algoritmo: A* | Criterio: transbordos
----------------------------------------------------------------------
1. En Aguablanca: Aborde la ruta T57 (espera/transbordo 4 min). Viaje 12 parada(s) hasta Santa Librada (24.8
    min).
     Paradas: Nuevo Latir -> Amanecer -> Troncal Unida -> Calipso -> Conquistadores -> Villanueva -> Santa
         Mónica -> Primitivo -> Cien Palos -> San Pascual -> San Bosco -> Santa Librada
2. En Santa Librada: Haga transbordo a la ruta T31 (espera/transbordo 9 min). Viaje 13 parada(s) hasta Univalle
    (25.3 min).
     Paradas: Manzana del Saber -> Estadio -> Tequendama -> Lido -> Unidad Deportiva -> Guadalupe -> Pampalinda
         -> Refugio -> Caldas -> Capri -> Meléndez -> Buitrera -> Univalle
Tiempo total estimado: 63.1 min | Transbordos: 1 | Paradas recorridas: 25
Nodos expandidos: 145 | generados: 193 | 2.07 ms
```

E5. Rutas suspendidas: alternativa sin transbordos

Menga -> Universidades con E21 y T31 suspendidas: el sistema encuentra la troncal T12 por el oriente.

```
$ python main.py ruta menga universidades --suspender e21 t31

Mejor ruta de Menga a Universidades
Algoritmo: A* | Criterio: tiempo
Rutas suspendidas: e21, t31
----------------------------------------------------------------------
1. En Menga: Aborde la ruta T12 (espera/transbordo 4 min). Viaje 26 parada(s) hasta Universidades (53 min).
     Paradas: Los Guaduales -> Calima -> Los Alcázares -> Metropolitano del Norte -> San Luis -> La Rivera ->
         Jorge Eliécer Gaitán -> Alfonso López -> Los Pinos -> Las Ceibas -> Nueva Base -> Ulpiano Lloreda ->
         Villa del Lago -> Lleras Restrepo -> Nueva Floresta -> El Pondaje -> Calipso -> El Diamante -> Antonio
         Nariño -> Ciudad Modelo -> República de Israel -> Villa del Sur -> Mariano Ramos -> Cañaverales ->
         Simón Bolívar -> Universidades
Tiempo total estimado: 57 min | Transbordos: 0 | Paradas recorridas: 26
Nodos expandidos: 36 | generados: 47 | 0.40 ms
```

E6. Estación sin servicio

Se suspenden T40 y T47: Andrés Sanín queda sin rutas y el sistema lo informa.

```
$ python main.py ruta andres_sanin centro --suspender t40 t47

Mejor ruta de Andrés Sanín a Centro
Algoritmo: A* | Criterio: tiempo
Rutas suspendidas: t40, t47
----------------------------------------------------------------------
No existe ruta entre las estaciones indicadas con las condiciones actuales.
Nodos expandidos: 1 | generados: 1 | 0.01 ms
```

E7. Consulta lógica directa a la base de conocimiento

¿A qué rutas se puede transbordar en Centro desde la T31?

```
$ python main.py consultar "transbordo(centro, t31, R)"

R = e37
R = t62
R = t50
R = t40
R = e21

(5 respuesta(s))
```

E8. Explicación (árbol de prueba) de un hecho derivado

¿Por qué con San Pascual cerrada el T40 va de Belalcázar directo a Centro?

```
$ python main.py explicar "disponible(t40, belalcazar, centro, T)" --cerrar san_pascual

disponible(t40, belalcazar, centro, 4.6)   <- R4: disponible(R, A, C, T) :- conecta(R, A, B, T1), cerrada(B),
    conecta(R, B, C, T2), distinto(A, C), \+ cerrada(A), \+ cerrada(C), \+ suspendida(R), suma(T1, T2, T).
   conecta(t40, belalcazar, san_pascual, 2.3)   <- R1: conecta(R, A, B, T) :- tramo(R, A, B, T).
      tramo(t40, belalcazar, san_pascual, 2.3)   [hecho]
   cerrada(san_pascual)   [hecho]
   conecta(t40, san_pascual, centro, 2.3)   <- R1: conecta(R, A, B, T) :- tramo(R, A, B, T).
      tramo(t40, san_pascual, centro, 2.3)   [hecho]
```

E9. Reglas cargadas y estadísticas de inferencia

```
$ python main.py reglas

Reglas de la base de conocimiento:

  R1   conecta(R, A, B, T) :- tramo(R, A, B, T).
  R2   sirve(R, E) :- conecta(R, E, _, _).
  R3   disponible(R, A, B, T) :- conecta(R, A, B, T), \+ cerrada(A), \+ cerrada(B), \+ suspendida(R).
  R4   disponible(R, A, C, T) :- conecta(R, A, B, T1), cerrada(B), conecta(R, B, C, T2), distinto(A, C), \+
      cerrada(A), \+ cerrada(C), \+ suspendida(R), suma(T1, T2, T).
  R5   opera_en(R, E) :- disponible(R, E, _, _).
  R6   transbordo(E, R1, R2) :- opera_en(R1, E), opera_en(R2, E), distinto(R1, R2).
  R7   estacion_transbordo(E) :- transbordo(E, _, _).
  R8   costo_transbordo(E, 3) :- estacion_transbordo(E), terminal(E).
  R9   costo_transbordo(E, 5) :- estacion_transbordo(E), \+ terminal(E).
  R10  espera(R, W) :- frecuencia(R, F), mitad(F, W).
  R11  vecinas(A, B) :- disponible(_, A, B, _).
  R12  alcanzable(A, B) :- vecinas(A, B).
  R13  alcanzable(A, C) :- vecinas(A, B), alcanzable(B, C).

Hechos base: 496 | Reglas: 13 | Hechos derivados: 8217 | Total: 8713
```

5. Análisis de resultados

• **Optimalidad:** A* obtuvo exactamente el mismo tiempo que costo uniforme (Dijkstra) en los 6480 pares origen-destino posibles, lo que confirma que la heurística es admisible.

• **Eficiencia:** sumando los 6480 pares, A* expandió 394 703 nodos frente a 579 500 de costo uniforme (32 % menos), porque la heurística (distancia en línea recta dividida entre la velocidad máxima de la red) orienta la búsqueda hacia el destino.

• **Estrategias no informadas o no óptimas:** la búsqueda voraz es la que menos nodos expande, pero puede devolver rutas mucho más lentas (88.4 min frente a 51.3 min en el escenario E2). La búsqueda en amplitud minimiza el número de acciones, no el tiempo, por lo que tampoco garantiza la mejor ruta.

• **Conocimiento dinámico:** al agregar hechos como *cerrada(san_pascual)* o *suspendida(t40)* el motor vuelve a inferir y la búsqueda se adapta sin modificar el código, lo que muestra la ventaja de separar la base de conocimiento del mecanismo de razonamiento.

• **Explicabilidad:** el comando *explicar* reconstruye el árbol de prueba de cualquier hecho derivado, indicando la regla aplicada en cada nivel.